

<스톱워치 / 시한폭탄>

팀명 : 홍박사와 아이들

팀원 : 박규현, 송성혁, 임송주, 홍순현

1. 프로그램 개요

본 프로그램은 ATtiny 계열 AVR MCU(클럭 1MHz 기준)를 사용하여 4자리 TM1650 7세그먼트 모듈과 부저, LED를 제어하는 스톱워치 + 시한폭탄 타이머 시스템이다.

MODE_STOPWATCH

- 일반 스톱워치 기능
- 0.1초 단위로 증가, 최대 9:59.9 (5999 데시초)까지 표시

MODE_BOMBTIMER

- 시한폭탄 타이머 기능
- 사용자가 분/초/0.1초 프리셋을 설정
- 남은 시간에 따라 부저 비프 주기와 길이가 점점 빨라지며, 0초 도달 시 LED와 폭발 멜로디를 재생
- TM1650의 4개 버튼을 이용하여 모드 전환, 시작/일시정지, 리셋 및 시간 설정을 수행한다. 시스템 시간 기준은 Timer0(1ms 타이머 인터럽트)로 관리하며, 부저 출력은 Timer1 PWM으로 구현.

2. 상수 및 공통 설정

TIME_DS_MAX

- TM1650 표시 형식이 MM:SS.d 이므로 최대 값은 9:59.9 $\rightarrow 9*600 + 59*10 + 9 = 5999$ 데시초로 제한.

STOPWATCH_TICK_MS, BOMB_TICK_MS

- 0.1초 단위를 사용하기 위해 100ms마다 1 데시초씩 증가/감소하도록 TICK 주기를 설정.

DOT_BLINK_MS, BOMB_IDLE_BLINK_MS

- 디스플레이 도트 및 자리 깜빡임 주기(500ms / 300ms)를 정의하여 사용자에게 상태를 직관적으로 보여줌.

BOMB_BEEP_ 상수들..

- 시한폭탄 모드에서 남은 시간에 따라 부저 비프 간격을 다르게 하기 위한 상수.

10초 초과: 2초 간격

5~10초: 1초 간격

3~5초: 0.5초 간격

1~3초: 0.25초 간격

1초 이하: 0.125초 간격 (짧은 비프)

```
// =====
//                                     공통 설정 / 상수
// =====

// TM1650 시간 표현: 최대 9:59.9 → 5999 데시초
#define TIME_DS_MAX      5999U

// 타이머 주기
#define TICK_1MS_OCR0A    124      // 1MHz / 8 / (124+1) = 1000Hz → 1ms
#define STOPWATCH_TICK_MS 100U     // 0.1초 = 100ms
#define BOMB_TICK_MS      100U     // 0.1초 = 100ms

// 도트/깜빡임 주기
#define DOT_BLINK_MS      500U     // 0.5초
#define BOMB_IDLE_BLINK_MS 300U

// 폭탄 부저 규칙
#define BOMB_BEEP_INTERVAL_LONG_MS 2000U // 10초 초과
#define BOMB_BEEP_INTERVAL_MID_MS  1000U // 5~10초
#define BOMB_BEEP_INTERVAL_SHORT_MS 500U  // 1~5초
#define BOMB_BEEP_INTERVAL_3S_MS   250U   // 1~3초
#define BOMB_BEEP_INTERVAL_1S_MS   125U   // 1초 이하
#define BOMB_BEEP_PULSE_MS          100U
#define BOMB_BEEP_PULSE_1S_MS       50U

// 부저 기본 주파수 (시한폭탄 카운트용)
#define BUZZER_BASE_FREQ_HZ 4000U
```

3. 소프트웨어 I2C 및 TM1650 제어부

3.1 I2C 비트뱅잉 저수준 함수

```
// =====
//                                     I2C (소프트웨어) / TM1650
// =====

#define SDA_PORT PORTB
#define SDA_DDR  DDRB
#define SDA_PINR PINB
#define SDA_BIT  PB0

#define SCL_PORT PORTB
#define SCL_DDR  DDRB
#define SCL_BIT  PB2
```

TM1650과 통신하기 위해 하드웨어 TWI 대신 소프트웨어 I2C(Bit-banging)를 사용.
SDA, SCL 핀은 DDR을 통해 입력(Hi-Z) = High, 출력 Low = 0 방식으로 제어한다.

static void i2c_sda_high(void);

-SDA를 입력(Hi-Z)로 설정하여 풀업에 의해 High 상태로 만듦.

static void i2c_sda_low(void);

-SDA를 출력으로 설정하고 Low를 출력.

static void i2c_scl_high(void); / i2c_scl_low(void);

-SCL 라인을 같은 방식으로 제어.

static void i2c_start(void);

-SCL High 상태에서 SDA를 High→Low로 전환하여 I2C START 조건 생성.

static void i2c_stop(void);

-SCL High 상태에서 SDA를 Low→High로 전환하여 I2C STOP 조건 생성.

static void i2c_write_bit(uint8_t bit);

-SCL 상승 에지 동안 SDA에 원하는 비트를 출력.

static uint8_t i2c_read_bit(void);

-SDA를 릴리즈한 상태에서 SCL를 High로 만들어 슬레이브가 구동하는 값을 읽음.

static void i2c_write_byte(uint8_t data);

상위 비트부터 1바이트를 전송하고, 마지막에 슬레이브의 ACK 비트를 읽음.

static uint8_t i2c_read_byte(uint8_t ack);

8비트를 읽은 뒤, 다음 바이트 수신 여부에 따라 ACK/NACK 비트를 출력.

3.2 TM1650 상위 제어 함수

-TM1650는 7세그먼트 디스플레이와 버튼 입력을 동시에 제공하는 IC임.

컨트롤/키 입력 주소: TM1650_CTRL_ADDR

각 자리 디지트 주소: TM1650_DIG1_ADDR~TM1650_DIG4_ADDR

주요 함수:

static void tm1650_write(uint8_t addr, uint8_t data);

-I2C START → (addr<<1)|0 → data → STOP 순으로 TM1650에 1바이트를 기록.

static uint8_t tm1650_read_key_raw(void);

```
// ----- TM1650 상위 함수 -----

// TM1650 주소 (7비트)
#define TM1650_CTRL_ADDR 0x24 // 컨트롤 + 키 입력
#define TM1650_DIG1_ADDR 0x34
#define TM1650_DIG2_ADDR 0x35
#define TM1650_DIG3_ADDR 0x36
#define TM1650_DIG4_ADDR 0x37

// 7세그 숫자 테이블 (DP 제외)
static const uint8_t seg_table[10] = {
    0x3F, // 0
    0x06, // 1
    0x5B, // 2
    0x4F, // 3
    0x66, // 4
    0x6D, // 5
    0x7D, // 6
    0x07, // 7
    0x7F, // 8
    0x6F, // 9
};
```

-컨트롤 주소에 read 모드로 접근하여 현재 키 코드(raw)를 읽음.

static int8_t tm1650_get_button_1_to_4(void);

-특정 키 코드(0xE4, 0xDC, 0xD4, 0xCC)를 1~4번 버튼으로 매핑.

-어떤 버튼도 눌리지 않은 경우 0을 반환.

static void tm1650_init(void);

-SDA/SCL 초기 상태 설정 및 딜레이 후, TM1650 컨트롤 레지스터를 0x87로 설정하여 디스플레이 ON

-적절한 밝기 설정

-초기 화면을 "0000"으로 출력.

4. 시간 디코딩 및 디스플레이 출력부

4.1 시간 디코딩

```
// ds(0.1초)를 4자리 7세그 코드로 변환
static void tm1650_decode_time_ds(uint16_t ds,
uint8_t *d1,
uint8_t *d2,
uint8_t *d3,
uint8_t *d4) {
    if (ds > TIME_DS_MAX) ds = TIME_DS_MAX;

    uint8_t minutes = ds / 600; // 60s = 600ds
    uint8_t tens_sec = (ds / 100) % 6; // 10초 자리 (0~5)
    uint8_t sec = (ds / 10) % 10; // 1초 자리
    uint8_t tenth = ds % 10; // 0.1초

    *d1 = seg_table[minutes];
    *d2 = seg_table[tens_sec];
    *d3 = seg_table[sec];
    *d4 = seg_table[tenth];
}
```

입력: ds (데시초 단위, 0.1초)

출력: 4자리 7세그 코드(d1~d4)

계산 방식:

minutes = ds / 600 (60.0초 = 600데시초)

tens_sec = (ds / 100) % 6 (10초 자리)

sec = (ds / 10) % 10 (1초 자리)

tenth = ds % 10 (0.1초)

-각 자리 숫자를 seg_table[]을 통해 7세그 패턴으로 변환.

4.2 일반 표시 / 깜빡임 표시

static void tm1650_display_time_ds(uint16_t ds, uint8_t dot_on);

-tm1650_decode_time_ds로 디코딩한 후, dot_on이 1이면 초(3번째 자리)에 DP 비트를 OR 하여 MM:SS.d 형태로 표시.

static void tm1650_display_time_ds_with_blink(...);

-프리셋 설정 시 특정 자리만 깜빡이기 위해 사용.

-selected_digit 자리의 값을 0x00으로 만들어 해당 자리를 꺼진 것처럼 표시.

5. Timer0 기반 1ms 시스템 타이머

```
// =====  
//                               Timer0 (1ms 시스템 타이머)  
// =====  
  
volatile uint32_t g_millis = 0;  
  
ISR(TIMER0_COMPA_vect) {  
    g_millis++;  
}
```

-Timer0를 CTC 모드로 설정하고, OCR0A = 124로 맞추어 $1\text{MHz} / 8 / (124+1) = 1000\text{Hz}$
→ 1ms 주기의 인터럽트를 생성.

-인터럽트 시마다 g_millis를 1씩 증가시키며, 시스템 공용 시각으로 사용.

static uint32_t millis_read(void);

-인터럽트 중단(cli() / sei())을 통해 g_millis를 안전하게 읽어옴.

-메인 루프와 인터럽트 사이의 충돌 방지하기 위한 처리.

6. 부저 제어 (Timer1 PWM)

6.1 기본 제어

```
void buzzer_set_frequency(uint16_t frequency_hz) {
    if (frequency_hz == 0) {
        buzzer_stop();
        return;
    }

    // Timer1 정지/초기화
    TCCR1 = 0;
    GTCCR = 0;
    TCNT1 = 0;

    // 1MHz / 1 / (OCR1C+1) = f
    uint32_t top32 = (F_CPU / frequency_hz) - 1;
    if (top32 > 255) top32 = 255;
    uint8_t top = (uint8_t)top32;

    OCR1C = top;
    OCR1B = top / 2; // 50% duty

    // PWM1B enable, OC1B non-inverting
    GTCCR = (1 << PWM1B) | (1 << COM1B1);

    // PB4 출력
    BUZZER_DDR |= (1 << BUZZER_PIN);

    // 분주 1
    TCCR1 = (1 << CS10);
}

void buzzer_stop(void) {
    TCCR1 = 0;
    GTCCR = 0;
    BUZZER_PORT &= ~(1 << BUZZER_PIN);
}
```

- Timer1을 사용하여 PB4(OC1B 핀)에 PWM 파형을 출력.
- OCR1C를 TOP, OCR1B를 듀티(50%)로 설정하여 원하는 주파수를 만들.
- top = F_CPU / f - 1 (단, 8bit 제한으로 255 이상 시 255로 제한)

6.2 비차단형 부저 펄스/연속 모드

```
// 비차단형 부저 제어용 상태
static uint8_t buzzer_active = 0;
static uint8_t buzzer_continuous = 0;
static uint32_t buzzer_off_deadline = 0;
```

buzzer_pulse_start(now_ms, length_ms);

- 현재 시각 기준으로 length_ms 동안만 부저를 울리는 단발 펄스를 시작.

buzzer_set_continuous(uint8_t on);

- 연속 울림 모드 ON/OFF 설정.

buzzer_update(now_ms);

-메인 루프에서 호출되며, 현재 시간이 buzzer_off_deadline을 지났을 경우 펄스를 자동으로 정지.

-이 방식으로 딜레이 없이도 부저 타이밍을 제어 가능.

7. 폭발 멜로디 처리

```
// =====  
//                               폭발 멜로디  
// =====  
  
typedef enum {  
    MELODY_NONE = 0,  
    MELODY_PLAYING  
} MelodyState;
```

폭탄이 0초에 도달했을 때 짧은 멜로디를 재생.

melody_freqs[], melody_durs[] 배열에 음높이(Hz)와 길이(ms)를 순서대로 정의해놓음.

주요 함수:

static void melody_start(uint32_t now_ms);

-상태를 MELODY_PLAYING으로 설정하고, 첫 노트부터 재생할 준비를 한다.

static void melody_update(uint32_t now_ms);

-현재 시각과 melody_step_deadline을 비교하여 노트 시간이 지났으면 다음 노트로 넘어가고, 주파수가 0인 경우는 침표로 처리해 부저를 끈다.

-모든 노트를 재생하면 MELODY_NONE 상태로 전환하고 부저를 멈춘다.

-멜로디 역시 비차단 방식으로, 메인 루프에서 주기적으로 melody_update()만 호출하면 재생이 진행.

8. LED 제어

```
// =====  
//                               LED  
// =====  
  
#define LED1_PIN PB3    // 시한폭탄 완료/경고 표시  
#define LED2_PIN PB1    // 스톱워치 동작 표시
```

-led_init()에서 두 핀을 출력으로 설정하고 초기에는 OFF.

- LED1은 시한폭탄 모드 표시,
- LED2는 스톱워치 표시용으로 사용.

9. 버튼 이벤트 플래그 및 처리 구조

```
// =====
//                      버튼 이벤트 플래그
// =====

volatile uint8_t btn1_clicked = 0;
volatile uint8_t btn2_clicked = 0;
volatile uint8_t btn3_clicked = 0;
volatile uint8_t btn4_clicked = 0;
```

- TM1650에서 읽은 버튼 값은 메인 루프에서 폴링한다.
- last_button_raw와 비교해서 0 → 1~4로 변화하는 옛지만 클릭으로 인식하고, 해당 버튼의 *_clicked 플래그를 1로 세팅.
- 이후 각 모드/기능 처리 함수에서 이 플래그를 읽어 동작을 수행한 뒤, 0으로 클리어.

10. 모드 및 상태 변수

```
// =====
//                      모드 / 상태
// =====

typedef enum {
    MODE_STOPWATCH = 0,
    MODE_BOMBTIMER = 1
} Mode;

typedef enum {
    BOMB_IDLE = 0, // 프리셋 설정 상태
    BOMB_RUNNING,
    BOMB_PAUSED,
    BOMB_DONE
} BombState;

typedef enum {
    SW_STATE_RESET = 0,
    SW_STATE_RUNNING,
    SW_STATE_PAUSED
} StopwatchState;
```

```
static Mode g_mode = MODE_STOPWATCH;
static BombState g_bomb_state = BOMB_IDLE;
static StopwatchState g_sw_state = SW_STATE_RESET;

// 데시초 (0.1초 단위)
static uint16_t stopwatch_ds = 0;
static uint16_t bomb_ds = 300; // 기본값: 30.0초
static uint16_t bomb_preset_ds = 300;

static uint8_t bomb_selected_digit = 0; // 0:분, 1:10초, 2:1초, 3:0.1초
```

- g_mode : 전체 모드 (스톱워치 / 폭탄 타이머)
- g_bomb_state : 폭탄 서브 상태 (설정 / 동작 / 일시정지 / 종료)
- g_sw_state : 스톱워치 상태 (리셋 / 동작 / 일시정지)

시간 값:

stopwatch_ds : 스톱워치 시간 (데시초)

bomb_ds : 폭탄 현재 남은 시간 (데시초)

bomb_preset_ds : 폭탄 프리셋 시간 (데시초)

11. 모드 전환 처리 (버튼1)

```
static void handle_mode_button(void) {
    if (!btn1_clicked) return;

    btn1_clicked = 0;
    uint32_t now = millis_read();

    if (g_mode == MODE_STOPWATCH) {
        // STOPWATCH → BOMBTIMER 전환
        // 폭탄이 이미 RUNNING 상태였다면, STOPWATCH 모드에 있는 동안 지난 시간을
        // 한 번에 반영하여 시간을 점프시킴 (숨은 카운트 유지)
        if (g_bomb_state == BOMB_RUNNING) {
            uint32_t elapsed_ms = now - last_bomb_tick_ms;
            uint16_t elapsed_ds = elapsed_ms / BOMB_TICK_MS;

            if (elapsed_ds > 0) {
                if (elapsed_ds ≥ bomb_ds) {
                    bomb_ds = 0;
                } else {
                    bomb_ds -= elapsed_ds;
                }
                // 기준 시각 재조정 (100ms 격자 맞추기)
                last_bomb_tick_ms = now - (elapsed_ms % BOMB_TICK_MS);
            }
        }

        g_mode = MODE_BOMBTIMER;

        led1_on();          // 폭탄 모드 표시
        led2_off();
        buzzer_set_continuous(0);
    } else {
        // BOMBTIMER → STOPWATCH 전환
        g_mode = MODE_STOPWATCH;

        led2_on();          // 스톱워치 모드 표시
        led1_off();
        buzzer_set_continuous(0);
    }

    dot_on = 1;
    last_dot_toggle_ms = now;
}
```

-버튼1 클릭 시 호출.

STOPWATCH → BOMBTIMER 전환 시:

-만약 폭탄이 이미 BOMB_RUNNING 상태였다면, 스톱워치 모드에 있는 동안 숨겨진 카운트 다운 시간을 한 번에 bomb_ds에 반영한다.

elapsed_ms = now - last_bomb_tick_ms;

`elapsed_ds = elapsed_ms / BOMB_TICK_MS;`

-그만큼 `bomb_ds`에서 빼고, 0 이하가 되면 0으로 고정.

즉, 폭탄 모드는 화면에 보이지 않을 뿐 실제 카운트는 계속 진행되도록 구현.

BOMBTIMER → STOPWATCH 전환 시:

-스톱워치 모드로 돌아가 LED2를 켜고 LED1을 끄며, 부저는 정지.

-두 모드 전환마다 `dot_on`과 `last_dot_toggle_ms`를 초기화하여 깜빡임 패턴이 깔끔하게 다시 시작되도록 함.

12. 스톱워치 동작 (MODE_STOPWATCH)

12.1 버튼 동작

```
// =====  
//                               스톱워치 처리  
//  
// - 모드: MODE_STOPWATCH  
// - 버튼2: start/pause  
// - 버튼3: reset  
// =====  
  
static void handle_stopwatch_buttons(uint32_t now_ms) {  
    if (btn2_clicked) {  
        btn2_clicked = 0;  
  
        if (g_sw_state == SW_STATE_RUNNING) {  
            g_sw_state = SW_STATE_PAUSED;  
        } else {  
            // reset 또는 paused → running  
            g_sw_state = SW_STATE_RUNNING;  
            last_sw_tick_ms = now_ms;  
        }  
    }  
  
    if (btn3_clicked) {  
        btn3_clicked = 0;  
        g_sw_state = SW_STATE_RESET;  
        stopwatch_ds = 0;  
    }  
}
```

버튼2: Start / Pause

RESET 또는 PAUSED → RUNNING (start)

RUNNING → PAUSED (일시정지)

버튼3: Reset

어떤 상태에서든 스톱워치를 RESET 상태로 만들고 시간을 0으로 만들.

12.2 시간 업데이트 및 표시

```
static void update_stopwatch(uint32_t now_ms) {
    if (g_mode != MODE_STOPWATCH) return;

    handle_stopwatch_buttons(now_ms);

    if (g_sw_state == SW_STATE_RESET) {
        stopwatch_ds = 0;
        led2_off();
    } else if (g_sw_state == SW_STATE_RUNNING) {
        led2_on();

        // 0.1초마다 데시조 1 증가
        while ((int32_t)(now_ms - last_sw_tick_ms) >= (int32_t)STOPWATCH_TICK_MS) {
            last_sw_tick_ms += STOPWATCH_TICK_MS;
            if (stopwatch_ds < TIME_DS_MAX) {
                stopwatch_ds++;
            }
        }
    } else { // PAUSED
        led2_off();
    }

    // 스톱워치 도트: 0.5초마다 깜빡
    if ((int32_t)(now_ms - last_dot_toggle_ms) >= (int32_t)DOT_BLINK_MS) {
        last_dot_toggle_ms += DOT_BLINK_MS;
        dot_on = !dot_on;
    }

    tm1650_display_time_ds(stopwatch_ds, dot_on);
}
```

g_sw_state에 따라 다음과 같이 처리.

RESET: 시간 0 유지

RUNNING: now_ms - last_sw_tick_ms >= 100ms마다 stopwatch_ds++ (TIME_DS_MAX 제한)

PAUSED: 시간 유지

-도트(초 자리 DP)는 500ms마다 토글하여 깜빡이는 효과를 줌.

-마지막에 tm1650_display_time_ds(stopwatch_ds, dot_on);으로 디스플레이를 갱신.

13. 시한폭탄 타이머 동작 (MODE_BOMBTIMER)

13.1 프리셋 자리 증가

```
// ===== 시한폭탄 처리 =====  
//  
// - 모드: MODE_BOMBTIMER  
// - 상태: BOMB_IDLE / RUNNING / PAUSED / DONE  
// - 버튼2: start / pause / resume / (DONE 에서) reset  
// - 버튼3: 선택 자리 값 증가 (IDLE/PAUSED 에서만)  
// - 버튼4: 자리 선택 (0~3 순환)  
// =====  
  
// 프리셋 시간에서 선택된 자리 증가  
static void bomb_increase_selected_digit(void) {  
    uint16_t ds = bomb_preset_ds;  
  
    uint8_t minutes = ds / 600;  
    uint8_t tens_sec = (ds / 100) % 6;  
    uint8_t sec = (ds / 10) % 10;  
    uint8_t tenth = ds % 10;  
  
    switch (bomb_selected_digit) {  
        case 0: // 분 (0~9)  
            minutes = (minutes + 1) % 10;  
            break;  
        case 1: // 10초 (0~5)  
            tens_sec = (tens_sec + 1) % 6;  
            break;  
        case 2: // 1초 (0~9)  
            sec = (sec + 1) % 10;  
            break;  
        case 3: // 0.1초 (0~9)  
            tenth = (tenth + 1) % 10;  
            break;  
        default:  
            break;  
    }  
  
    bomb_preset_ds = minutes * 600 + tens_sec * 100 + sec * 10 + tenth;  
    if (bomb_preset_ds > TIME_DS_MAX) bomb_preset_ds = TIME_DS_MAX;  
}
```

-bomb_preset_ds 기준으로 현재 선택된 자리(bomb_selected_digit)의 값을 1씩 증가.

자리별 범위:

분: 0~9

10초: 0~5

1초: 0~9

0.1초: 0~9

-변경 후 다시 데시초로 합산하여 bomb_preset_ds에 저장하고, 범위를 TIME_DS_MAX로 제한.

13.2 폭탄 버튼 처리 (버튼2/3/4)

```
// 버튼2/3/4 처리
static void handle_bomb_buttons(void) {
    if (btn2_clicked) {
        btn2_clicked = 0;

        if (g_bomb_state == BOMB_IDLE) {
            // 처음 시작
            bomb_ds = bomb_preset_ds;
            g_bomb_state = BOMB_RUNNING;
            last_bomb_tick_ms = millis_read();
            last_bomb_beep_ms = last_bomb_tick_ms;
        } else if (g_bomb_state == BOMB_RUNNING) {
            // 일시정지
            g_bomb_state = BOMB_PAUSED;
            buzzer_set_continuous(0);
        } else if (g_bomb_state == BOMB_PAUSED) {
            // 다시 시작
            g_bomb_state = BOMB_RUNNING;
            last_bomb_tick_ms = millis_read();
            last_bomb_beep_ms = last_bomb_tick_ms;
        } else if (g_bomb_state == BOMB_DONE) {
            // DONE → 다시 IDLE 상태로 초기화
            g_bomb_state = BOMB_IDLE;
            bomb_ds = bomb_preset_ds;
            buzzer_set_continuous(0);
            led1_off();
        }
    }

    if (btn3_clicked) {
        btn3_clicked = 0;
        if (g_bomb_state == BOMB_IDLE || g_bomb_state == BOMB_PAUSED) {
            bomb_increase_selected_digit();
            bomb_ds = bomb_preset_ds;
        }
    }

    if (btn4_clicked) {
        btn4_clicked = 0;
        if (g_bomb_state == BOMB_IDLE || g_bomb_state == BOMB_PAUSED) {
            bomb_selected_digit = ((uint8_t)((bomb_selected_digit + 1) & 0x03)); // 0~3 순환
        }
    }
}
```

버튼2

BOMB_IDLE → BOMB_RUNNING (bomb_ds를 프리셋값으로 세팅하고 카운트다운 시작)

BOMB_RUNNING → BOMB_PAUSED (일시정지, 부저 정지)

BOMB_PAUSED → BOMB_RUNNING (다시 카운트다운 시작)

BOMB_DONE → BOMB_IDLE 복귀, LED1과 부저 정지

버튼3

BOMB_IDLE 또는 BOMB_PAUSED 상태에서 선택 자리 값 증가

증가 후 bomb_ds = bomb_preset_ds로 현재 남은 시간도 동기화

버튼4

BOMB_IDLE 또는 BOMB_PAUSED에서 자리 선택 (분→10초→1초→0.1초 순환)

13.3 폭탄 카운트다운 및 표시

```
// 폭탄 타이머 전체 상태 업데이트 + 표시
static void update_bomb(uint32_t now_ms) {
    if (g_mode != MODE_BOMBTIMER) return;

    handle_bomb_buttons();

    // 카운트다운 (RUNNING 상태에서만)
    if (g_bomb_state == BOMB_RUNNING) {
        if ((int32_t)(now_ms - last_bomb_tick_ms) ≥ (int32_t)BOMB_TICK_MS) {
            last_bomb_tick_ms += BOMB_TICK_MS;
            if (bomb_ds > 0) {
                bomb_ds--;
            }
        }
    }

    // ==== 디스플레이 처리 ====

    if (g_bomb_state == BOMB_RUNNING) {
        // 러닝 중: 도트 0.5초 깜빡 + 남은 시간 표시
        if ((int32_t)(now_ms - last_dot_toggle_ms) ≥ (int32_t)DOT_BLINK_MS) {
            last_dot_toggle_ms += DOT_BLINK_MS;
            dot_on = !dot_on;
        }
        tm1650_display_time_ds(bomb_ds, dot_on);
    } else if (g_bomb_state == BOMB_DONE) {
        // DONE: 0.0 고정 (도트 ON)
        tm1650_display_time_ds(bomb_ds, 1);
    } else if (g_bomb_state == BOMB_PAUSED) {
        // PAUSED: 멈춘 시점의 남은 시간 표시 (도트 ON)
        tm1650_display_time_ds(bomb_ds, 1);
    } else {
        // BOMB_IDLE: 프리셋 설정 모드
        // 선택된 자리만 깜빡 (0.3초 주기)
        if ((int32_t)(now_ms - last_dot_toggle_ms) ≥ (int32_t)BOMB_IDLE_BLINK_MS) {
            last_dot_toggle_ms += BOMB_IDLE_BLINK_MS;
            dot_on = !dot_on;
        }

        tm1650_display_time_ds_with_blink(
            bomb_preset_ds,
            bomb_selected_digit,
            dot_on
        );
    }

    update_bomb_buzzer(now_ms);
}
```

BOMB_RUNNING 상태에서만 카운트다운 수행

now_ms - last_bomb_tick_ms >= 100ms마다 bomb_ds-- (음수로 내려가지 않도록 관리)

디스플레이 처리

RUNNING: 도트(DP)를 0.5초마다 깜빡이면서 bomb_ds를 표시.

DONE: 0.0 고정 표시, 도트 ON

PAUSED: 일시정지된 시점의 bomb_ds를 표시, 도트 ON

IDLE: bomb_preset_ds를 기반으로, 선택된 자리만 300ms 주기로 깜빡이도록 표시
(tm1650_display_time_ds_with_blink() 사용)

13.4 폭탄 부저 규칙

```
// 폭탄 부저 업데이트
static void update_bomb_buzzer(uint32_t now_ms) {
    // 멜로디 재생 중이면 멜로디만 갱신
    if (melody_state == MELODY_PLAYING) {
        melody_update(now_ms);
        return;
    }

    // 폭탄이 RUNNING 상태가 아니면, 규칙에 따른 비프 없음
    if (g_bomb_state != BOMB_RUNNING) {
        buzzer_set_continuous(0);
        return;
    }

    uint16_t ds = bomb_ds;
    uint32_t remain_ms = (uint32_t)ds * 1000U;

    // 0으로 떨어진 시점 → 폭발 처리 + 멜로디 시작
    if (ds == 0) {
        g_bomb_state = BOMB_DONE;
        led1_on();
        buzzer_set_continuous(0);
        melody_start(now_ms);
        return;
    }

    // 1초 이하: 종종하게 짧게 뻑
    if (remain_ms ≤ 1000U) {
        uint16_t interval_ms = BOMB_BEEP_INTERVAL_1S_MS;
        if ((int32_t)(now_ms - last_bomb_beep_ms) ≥ (int32_t)interval_ms) {
            last_bomb_beep_ms = now_ms;
            buzzer_pulse_start(now_ms, BOMB_BEEP_PULSE_1S_MS);
        }
        return;
    }

    // 3초 이하: 0.25초 간격
    if (remain_ms ≤ 3000U) {
        uint16_t interval_ms = BOMB_BEEP_INTERVAL_3S_MS;
        if ((int32_t)(now_ms - last_bomb_beep_ms) ≥ (int32_t)interval_ms) {
            last_bomb_beep_ms = now_ms;
            buzzer_pulse_start(now_ms, BOMB_BEEP_PULSE_MS);
        }
        return;
    }

    // 그 외 구간: 5/10초 기준 기존 규칙
    uint16_t interval_ms;
    if (remain_ms > 10000U) { // 10초 초과
        interval_ms = BOMB_BEEP_INTERVAL_LONG_MS;
    } else if (remain_ms > 5000U) { // 5~10초
        interval_ms = BOMB_BEEP_INTERVAL_MID_MS;
    } else { // 3~5초
        interval_ms = BOMB_BEEP_INTERVAL_SHORT_MS;
    }

    if ((int32_t)(now_ms - last_bomb_beep_ms) ≥ (int32_t)interval_ms) {
        last_bomb_beep_ms = now_ms;
        buzzer_pulse_start(now_ms, BOMB_BEEP_PULSE_MS);
    }
}
```

-폭탄 상태가 BOMB_RUNNING일 때 남은 시간 bomb_ds를 바탕으로 위에서 정의한 규칙에 따라 비프 간격을 조절.

-남은 시간을 $\text{remain_ms} = \text{ds} * 100$ 으로 계산하고, 범위별로 다른 interval 사용.

1초 이하: BOMB_BEEP_INTERVAL_1S_MS, BOMB_BEEP_PULSE_1S_MS

1~3초: BOMB_BEEP_INTERVAL_3S_MS

3~5초: BOMB_BEEP_INTERVAL_SHORT_MS

5~10초: BOMB_BEEP_INTERVAL_MID_MS

10초 초과: BOMB_BEEP_INTERVAL_LONG_MS

-조건에 맞는 간격이 지날 때마다 buzzer_pulse_start()를 호출하여 짧은 비프를 발생.

-ds == 0이 되는 순간 연속 부저 OFF 후 melody_start() 호출로 폭발 멜로디 재생

14. main 함수 동작 흐름

init

-TM1650, Timer0, LED 초기화 후 전역 인터럽트(sei()) 활성화

-디스플레이에 000.0을 표시

while

-millis_read()로 현재 시각 획득

-TM1650에서 버튼 값을 읽어 btnX_clicked 플래그 설정

-handle_mode_button()으로 모드 전환 처리

-현재 모드에 따라 스톱워치 또는 시한폭탄 갱신

-buzzer_update()로 부저 펄스 종료 타이밍 관리

~_delay_ms(10)를 통해 버튼 폴링 주기를 10ms로 제한

이와 같이 전체 코드는 1ms 기반 시스템 타이머, 소프트웨어 I2C를 이용한 TM1650 디스플레이 및 버튼 제어, Timer1 PWM을 이용한 부저 및 멜로디 재생, LED 상태 표시, 모드/상태 머신을 조합하여 스톱워치와 시한폭탄 타이머 기능을 동시에 제공하도록 구현하였음.

전체 회로도


```

// =====
//                               main
// =====

int main(void) {
    _delay_ms(10);

    tm1650_init();
    timer0_init();
    led_init();

    sei();

    // 디스플레이 초기값
    tm1650_display_time_ds(0, 1);

    uint8_t last_button_raw = 0; // 이전 TM1650 버튼 raw 값

    while (1) {
        uint32_t now = millis_read();

        // ----- TM1650 버튼 읽기 & 옯쳐-검출 -----
        int8_t btn = tm1650_get_button_1_to_4();

        if (btn != 0 && last_button_raw == 0) {
            // 0 + 1~4로 올라가는 순간만 클릭으로 인식
            switch (btn) {
                case 1: btn1_clicked = 1; break;
                case 2: btn2_clicked = 1; break;
                case 3: btn3_clicked = 1; break;
                case 4: btn4_clicked = 1; break;
                default: break;
            }
        }
        last_button_raw = (uint8_t)btn;

        // ----- 모드 전환 처리 -----
        handle_mode_button();

        // ----- 각 모드 동작 -----
        if (g_mode == MODE_STOPWATCH) {
            update_stopwatch(now);
        } else {
            update_bomb(now);
        }

        // 부저 펄스 폭 처리
        buzzer_update(now);

        // 버튼 채터링/잡음 주기
        _delay_ms(10);
    }
}

```

